

JS_Backend (Reelify)

System Design & Code Architecture Analysis

April 29, 2026

Contents

1	Project Overview	3
1.1	Purpose	3
1.2	Key Features	3
1.3	Tech Stack	3
1.4	High-Level Architecture Summary	3
2	System Architecture	3
2.1	Architecture Style	3
2.2	Core Components	3
2.3	Data Flow (Overview)	4
3	Code Structure Breakdown	4
3.1	Folder Structure and Responsibilities	4
3.2	Dependency Relationships (Layering)	4
4	Data Flow & Lifecycle	4
4.1	Request Flow (Step-by-Step)	4
4.2	App Bootstrapping	5
4.3	AppFactory Composition Root	5
5	Design Principles Used (Mapped to Code)	6
5.1	SOLID: Single Responsibility Principle (SRP)	6
5.2	SOLID: Open/Closed Principle (OCP)	6
5.3	SOLID: Liskov Substitution Principle (LSP)	7
5.4	SOLID: Dependency Inversion Principle (DIP)	7
5.5	DRY: Template Method in Base Classes	8
5.6	KISS: Explicit Composition	8
6	Design Patterns Used (Mapped to Code)	8
6.1	Singleton Pattern	8
6.2	Factory Method Pattern	9
6.3	Template Method Pattern	9
6.4	Strategy + Adapter Pattern (asyncHandler)	9
6.5	Observer Pattern (Mongoose pre-save hook / Error middleware)	10
6.6	Facade Pattern (UserModelService)	10
6.7	Adapter Pattern (mongooseAggregatePaginate Plugin)	10
7	Database Design & Schema	11
8	Advanced System Design Concepts	11
9	Q&A; Section	12
10	Strengths & Weaknesses	13

1 Project Overview

1.1 Purpose

JS_Backend (Reelify) is a TypeScript + Node.js + Express backend for a YouTube-like video-sharing platform. It handles user authentication (JWT + refresh tokens), video management, subscriptions, watch history, and a channel-profile aggregation pipeline. It is structured around a clean layered architecture with explicit OOP base classes to enforce consistency across all contributors.

1.2 Key Features

- **Auth:** Register, login, logout, refresh-token rotation, change-password, and optional-auth routes.
- **Users:** Avatar / cover-image upload via Cloudinary, channel-profile aggregation with subscriber counts.
- **Videos:** CRUD with file upload (Multer), view counting, and aggregation-based pagination.
- **Subscriptions:** Subscribe / unsubscribe, list subscribers and subscribed channels.
- **Recommendations:** Dedicated recommendation router.
- **Health:** /api/v1/health endpoint — used by KeepAliveService to prevent free-tier sleep.

1.3 Tech Stack

- **Runtime:** Node.js (ES modules), TypeScript 5.
- **Framework:** Express 4.
- **Database:** MongoDB via Mongoose 9.
- **Auth:** JWT (jsonwebtoken), bcrypt password hashing, HTTP-only cookies.
- **File Upload:** Multer (disk storage) + Cloudinary CDN.
- **Scheduler:** node-cron (KeepAliveService).
- **Build:** tsc → dist/, nodemon for dev.

1.4 High-Level Architecture Summary

The codebase follows a **layered, OOP-driven architecture** with abstract base classes providing shared contract enforcement across all routes, controllers, services, and models:

- **Presentation:** Express routers + controllers (HTTP boundary).
- **Service Layer:** UserModelService and BaseService wrapping Mongoose operations.
- **Infrastructure:** DatabaseService (Singleton), KeepAliveService (Singleton), Cloudinary utility.
- **Cross-cutting:** ApiError / ApiResponse value objects, asyncHandler adapter, error middleware.

2 System Architecture

2.1 Architecture Style

Layered monolith: a single deployable TypeScript service compiled to dist/. Not microservices; request processing is synchronous HTTP with async MongoDB I/O via Mongoose.

2.2 Core Components

- **src/index.ts:** Process entrypoint — orchestrates DB connect, AppFactory, KeepAlive.
- **src/app.ts (AppFactory):** Factory that builds and returns the configured Express Application.
- **src/base/:** Abstract base classes — BaseController, BaseRoute, BaseService, BaseError.
- **src/routes/:** Feature routers mounted under /api/v1/.
- **src/controllers/:** Request handlers using asyncHandler + ApiError/ApiResponse.
- **src/models/:** Mongoose schemas with instance/static methods (User, Video, Subscription, BaseModel).
- **src/services/:** DatabaseService (Singleton), UserModelService (Facade).
- **src/utills/:** ApiError, ApiResponse, asyncHandler, KeepAliveService, cloudinary, sendEmail.
- **src/middlewares/:** auth.middleware, errors.middleware, multer.middleware, optionalAuth.middleware.

2.3 Data Flow (Overview)

A request enters Express, passes through security + auth middleware, is routed to a controller function (wrapped in asyncHandler), which calls Mongoose model methods or the UserModelService, and responds with an ApiResponse object. Any thrown ApiError is caught by asyncHandler and forwarded to the global error middleware.

3 Code Structure Breakdown

3.1 Folder Structure and Responsibilities

- **src/index.ts:** Bootstrap — DB + AppFactory + KeepAlive.
- **src/app.ts:** AppFactory — middleware, routes, error-handler composition.
- **src/base/BaseController.ts:** Abstract — handleError(), sendSuccess() template methods.
- **src/base/BaseRoute.ts:** Abstract — enforces setupRoutes() via Template Method.
- **src/base/BaseService.ts:** Generic CRUD service with injected Mongoose Model.
- **src/base/BaseError.ts:** Extensible error hierarchy (LSP-correct prototype chain).
- **src/services/DatabaseService.ts:** Singleton MongoDB connection manager.
- **src/services/UserModelService.ts:** Facade over Mongoose User model.

- **src/utils/ApiError.ts / ApiResponse.ts**: Typed value-object wrappers for HTTP I/O.
- **src/utils/asyncHandler.ts**: Strategy + Adapter HOF — wraps async handlers for Express.
- **src/utils/KeepAliveService.ts**: Singleton cron-based health pinger.

3.2 Dependency Relationships (Layering)

- Routes depend on controllers and middleware (presentation layer).
- Controllers depend on Mongoose models directly or via service layer (UserModelService).
- Services (DatabaseService, UserModelService) depend on Mongoose abstractions.
- All layers depend on cross-cutting utilities: ApiError, ApiResponse, asyncHandler.
- BaseController / BaseRoute / BaseService are depended upon by all concrete implementations.

4 Data Flow & Lifecycle

4.1 Request Flow (Step-by-Step)

- 1. Startup**: DatabaseService.getInstance().connect() ensures one MongoDB connection.
- 2. App Build**: new AppFactory(config).getApp() registers middleware, routes, error-handler.
- 3. Middleware**: CORS, JSON parsing, cookieParser, static files.
- 4. Auth**: verifyJWT middleware attaches req.user on protected routes; optionalAuth for public+private.
- 5. Routing**: Feature routers dispatch to controller methods wrapped in asyncHandler.
- 6. Response**: Controllers throw ApiError (caught by asyncHandler → error middleware) or return new ApiResponse().

4.2 App Bootstrapping

```
// src/index.ts
async function startServer(): Promise<void> {
  // Singleton – one DB connection
  const dbService = DatabaseService.getInstance();
  await dbService.connect();

  // Factory – builds the Express app
  const appFactory = new AppFactory({ corsOrigin: CORS_ORIGIN });
  const app = appFactory.getApp();

  app.listen(PORT, '0.0.0.0', () => {
    // Singleton cron – prevent free-tier sleep
    const keepAlive = KeepAliveService.getInstance(SERVER_URL);
    keepAlive.start();
  });
}
```

4.3 AppFactory Composition Root

```
// src/app.ts
export class AppFactory {
  private readonly app: Application;

  constructor(private readonly config: AppConfig) {
    this.app = express();
    this.setupMiddlewares(); // SRP: one concern per method
    this.setupRoutes();      // OCP: add routes without touching middleware
    this.setupErrorHandling();
  }

  private setupRoutes(): void {
    this.app.use('/api/v1/users',      userRouter);
    this.app.use('/api/v1/videos',    videoRouter);
    this.app.use('/api/v1/subscriptions', subscriptionRouter);
    this.app.use('/api/v1/recommend', recommendationRouter);
    this.app.use('/api/v1/health',    healthRouter);
  }

  getApp(): Application { return this.app; }
}
```

Listing: src/app.ts — AppFactory

5 Design Principles Used (Mapped to Code)

5.1 SOLID: Single Responsibility Principle (SRP)

Explanation: Every class/module owns exactly one axis of change.

Why it matters: Limits the blast radius of a change — editing the error response shape should not touch authentication or route registration.

Where implemented

- **AppFactory:** three private methods each own one concern — middleware, routes, error-handler.
- **DatabaseService:** only manages the Mongoose connection lifecycle.
- **KeepAliveService:** only schedules periodic health pings.
- **asyncHandler:** only wraps async functions for Express compatibility.
- **ApiError / ApiResponse:** model error and success response shapes — nothing else.

```
// src/app.ts – each private method owns ONE concern
private setupMiddlewares(): void { /* cors, json, static */ }
private setupRoutes(): void      { /* route mounting only */ }
private setupErrorHandling(): void { this.app.use(errorHandler); }
```

Listing: SRP in AppFactory

5.2 SOLID: Open/Closed Principle (OCP)

Explanation: Classes are open for extension but closed for modification.

Why it matters: Adding a new route group, a new controller, or a new error type requires writing new code — not editing existing, tested code.

Where implemented

- **BaseRoute:** adding a new route file means extending BaseRoute and implementing setupRoutes() — BaseRoute itself never changes.
- **BaseService:** new services (e.g. VideoService) extend BaseService with model-specific logic without modifying generic CRUD.
- **BaseError** → **ApiError:** the error middleware handles any instance of BaseError uniformly — no code change needed when adding new error subtypes.

```
// src/base/BaseRoute.ts
export abstract class BaseRoute {
  public readonly router: Router;
  constructor() {
    this.router = Router();
    this.setupRoutes(); // Template Method hook – called at construction
  }
  abstract setupRoutes(): void; // Subclass EXTENDS without touching BaseRoute
}
```



```
// New route file – zero changes to BaseRoute
class HealthRoute extends BaseRoute {
  setupRoutes() { this.router.get('/', healthCheck); }
}
```

Listing: OCP in BaseRoute

5.3 SOLID: Liskov Substitution Principle (LSP)

Explanation: Subclasses must be substitutable for their parent without breaking callers.

Why it matters: TypeScript transpilation to ES5 breaks the prototype chain for custom Error subclasses — instanceof checks silently fail unless fixed explicitly.

Where implemented

- **BaseError:** calls `Object.setPrototypeOf(this, BaseError.prototype)` so that instanceof BaseError works correctly at runtime even after tsc compilation.
- **ApiError extends Error:** same fix — ensures the Express error middleware's instanceof ApiError check is reliable.

```
// src/base/BaseError.ts
export class BaseError extends Error {
  constructor(statusCode, message, errors = [], data = null, stack?) {
    super(message);
    this.statusCode = statusCode;
    this.success = false;
    // LSP FIX: correct prototype chain after TypeScript ES5 transpile
    Object.setPrototypeOf(this, BaseError.prototype);
    if (stack) { this.stack = stack; }
    else { Error.captureStackTrace(this, this.constructor); }
  }
}
```

Listing: LSP fix in BaseError.ts

5.4 SOLID: Dependency Inversion Principle (DIP)

Explanation: High-level modules should not depend on low-level details; both should depend on abstractions.

Why it matters: Swapping MongoDB for another DB, or replacing Mongoose, should not require rewriting controllers or route handlers.

Where implemented

- **index.ts:** calls `DatabaseService.getInstance().connect()` — never touches `mongoose.connect()` directly.
- **BaseService:** constructor accepts a Model abstraction; concrete services inject their own Mongoose model at construction time.

- **Controllers:** depend on the UserModelService abstraction, not on the User Mongoose model directly.

```
// src/base/BaseService.ts
export abstract class BaseService<T extends Document> {
  // DIP: accepts Model<T> abstraction – not a specific concrete model
  constructor(protected readonly model: Model<T>) {}

  async findById(id: string): Promise<T | null> {
    return await this.model.findById(id); // Delegates to injected abstraction
  }
}

// Concrete service injects the concrete model at instantiation time
class VideoService extends BaseService<IVideo> {
  constructor() { super(Video); } // only THIS line knows about 'Video'
}
```

Listing: DIP in BaseService.ts

5.5 DRY: Template Method in Base Classes

Explanation: A pattern that applies once (e.g. CRUD boilerplate, error handling, route registration) should have one canonical location.

Where implemented

- **BaseController.handleError() / sendSuccess():** every controller inherits the same response-building logic — no duplication across user, video, subscription controllers.
- **BaseService CRUD methods:** create, findById, updateById, deleteById are implemented once; all concrete services reuse without copying.

5.6 KISS: Explicit Composition Without Framework Magic

Explanation: Prefer explicit wiring over magic convention/reflection.

Where implemented

- **AppFactory:** explicitly calls setupMiddlewares(), setupRoutes(), setupErrorHandling() in the constructor — the order is readable and not inferred.
- **asyncHandler:** a single 5-line function instead of a decorator framework.
- **Manual DI** — no IoC container; dependencies are passed through constructors.

6 Design Patterns Used (Mapped to Code)

6.1 Singleton Pattern

Intent: Ensure only one instance of a class is ever created.

Where used: DatabaseService and KeepAliveService — both use private constructor + static getInstance().

Why chosen: Prevents duplicate MongoDB connections or multiple competing cron jobs.

```
// src/services/DatabaseService.ts
export class DatabaseService {
  private static instance: DatabaseService;
  private constructor() {} // prevents external 'new'

  static getInstance(): DatabaseService {
    if (!DatabaseService.instance) {
      DatabaseService.instance = new DatabaseService();
    }
    return DatabaseService.instance;
  }
}
```

Listing: Singleton in DatabaseService.ts

6.2 Factory Method Pattern

Intent: Encapsulate object creation — callers receive a product without knowing how it was built.

Where used

- **AppFactory.getApp():** returns a fully configured Express Application. `index.ts` never calls `express()` or `app.use()` directly.
- **User.generateAccessToken() / generateRefreshToken():** model instance methods act as factories that produce signed JWT strings.
- **mongoose.model(...):** the Mongoose factory call that produces the typed model proxy.

```
// src/app.ts
export class AppFactory {
  getApp(): Application {
    return this.app; // Caller gets the product without knowing build details
  }
}

// src/index.ts – Factory Method usage
const appFactory = new AppFactory({ corsOrigin: CORS_ORIGIN });
const app = appFactory.getApp(); // Receives configured app
```

Listing: Factory Method in AppFactory

6.3 Template Method Pattern

Intent: Define a skeleton algorithm in a base class; let subclasses fill in specific steps.

Where used

- **BaseRoute constructor:** calls `this.setupRoutes()` — concrete subclasses provide the body.
- **BaseController:** `handleError()` and `sendSuccess()` are reusable skeleton steps that all controllers call.
- **AppFactory constructor:** calls `setupMiddlewares()` → `setupRoutes()` → `setupErrorHandling()` in a fixed sequence (Template Method flavour).

```
// src/base/BaseRoute.ts
export abstract class BaseRoute {
  constructor() {
    this.router = Router();
    this.setupRoutes(); // Template: always called, always at construction
  }
  abstract setupRoutes(): void; // Step filled in by each concrete subclass
}

// src/base/BaseController.ts – skeleton response methods
protected handleError(error: unknown, res: Response): void {
  if (error instanceof ApiError) { res.status(error.statusCode).json({...}); }
  else { res.status(500).json({ message: 'Unknown error' }); }
}
```

Listing: Template Method in BaseRoute & BaseController

6.4 Strategy + Adapter Pattern (asyncHandler)

Intent (Strategy): Wrap any async handler as an interchangeable strategy with unified error-forwarding behaviour.

Intent (Adapter): Convert an async function (Promise-based) to the synchronous Express middleware signature.

Where used: src/utils/asyncHandler.ts — wraps every controller method.

```
// src/utils/asyncHandler.ts
export const asyncHandler = (requestHandler: AsyncRequestHandler) => {
  return (req: Request, res: Response, next: NextFunction): void => {
    // Adapter: converts async fn → sync Express callback
    // Strategy: error-forwarding is defined ONCE, applied to any handler
    Promise.resolve(requestHandler(req, res, next))
      .catch((error: Error) => next(error));
  };
};

// Usage in every controller:
const registerUser = asyncHandler(async (req, res) => { ... });
```

Listing: Strategy + Adapter in asyncHandler.ts

6.5 Observer Pattern (Mongoose pre-save hook + Error Middleware)

Intent: React to events without the emitter knowing about the subscriber.

Where used

- **userSchema.pre('save')**: Mongoose fires this hook before every save — the password is hashed automatically without any controller calling bcrypt explicitly.
- **errors.middleware.ts**: registered as the last Express handler — acts as a global subscriber to all next(error) calls emitted anywhere in the app.

```
// src/models/user.model.ts
// Observer: fires BEFORE every save – hashes password if modified
userSchema.pre<IUser>('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

// src/middlewares/errors.middleware.ts
// Observer: receives all next(error) calls from any route/controller
const errorHandler = (err, _req, res, _next) => {
  const statusCode = err instanceof ApiError ? err.statusCode : 500;
  res.status(statusCode).json({ success: false, message: err.message });
};
```

Listing: Observer in user.model.ts + errors.middleware.ts

6.6 Facade Pattern (UserModelService)

Intent: Provide a simplified interface over a complex subsystem.

Where used: `src/services/UserModelService.ts` — wraps Mongoose User model with intention-revealing methods so controllers never call `User.findOne()` with raw queries.

```
// src/services/UserModelService.ts
export class UserModelService {
  // Facade: hides '$or' query shape from callers
  async registerUser(userData: UserDTO): Promise<IUser> {
    const existing = await User.findOne({
      $or: [{ username: userData.username }, { email: userData.email }]
    });
    if (existing) throw new ApiError(409, 'User already exists');
    return await User.create(userData);
  }
  // Simple intention-revealing names hide Mongoose internals
  async findUserByEmail(email: string) { return User.findOne({ email }); }
  async clearRefreshToken(userId: string) {
    await User.findByIdAndUpdate(userId, { refreshToken: null });
  }
}
```

Listing: Facade in UserModelService.ts

6.7 Adapter Pattern (mongooseAggregatePaginate Plugin)

Intent: Allow incompatible interfaces to work together without modifying either.

Where used: `video.model.ts` and `user.model.ts` — the third-party `mongoose-aggregate-paginate-v2` plugin is adapted into the Mongoose schema via `schema.plugin()`, making aggregation pagination available without schema modification.

```
// src/models/video.model.ts
// Adapter: plugs third-party pagination API into Mongoose schema
videoSchema.plugin(mongooseAggregatePaginate);

// Now Video.aggregatePaginate(...) is available as if it were native Mongoose
```

Listing: Adapter Pattern in video.model.ts

7 Database Design & Schema

7.1 Design Choices

- **MongoDB + Mongoose:** document-oriented storage suits the denormalized video/user/subscription data.
- **UUID-equivalent:** MongoDB ObjectId primary keys used across all collections.
- **Indexes:** username (unique + index) and fullName (index) on User; enables fast lookups.
- **Relationships via references:** Video.owner → User ObjectId; Subscription.subscriber + .channel → User ObjectId (both with ref: 'User').
- **Watch history:** stored as an array of Video ObjectId refs on the User document.
- **Instance methods on models:** isPasswordCorrect(), generateAccessToken(), generateRefreshToken(), incrementViews() — domain logic lives close to the data.
- **Static query methods:** findByUsername(), findByEmail() on User; findPublished(), findByOwner() on Video — abstraction over raw filter objects.
- **Aggregate pagination:** mongooseAggregatePaginate plugin on User and Video enables cursor-based pagination for channel subscriber feeds and video listings.

```
// src/models/user.model.ts (schema excerpt)
const userSchema = new Schema<IUser>({
  username: { type: String, required: true, unique: true, index: true },
  email:    { type: String, required: true, unique: true },
  password: { type: String, required: true },    // stored hashed (bcrypt)
  refreshToken: { type: String },
  watchHistory: [{ type: Schema.Types.ObjectId, ref: 'Video' }],
  avatar:    { type: String, required: true },
  coverImage: { type: String },
}, { timestamps: true });

// src/models/video.model.ts (key fields)
const videoSchema = new Schema<IVideo>({
  owner: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  views: { type: Number, default: 0 },
  isPublished: { type: Boolean, default: true },
}, { timestamps: true });
```

Listing: MongoDB schemas (User + Video)

8 Advanced System Design Concepts (As Implemented)

8.1 Scalability (Practical Signals)

- **Mongoose connection pooling:** Mongoose maintains a connection pool internally; DatabaseService ensures only one pool is created (Singleton).
- **I/O concurrency:** async/await throughout; Node.js event loop handles concurrent requests.

- **Aggregation pipeline:** MongoDB \$lookup + \$addFields used in getUserChannelProfile — efficient join-equivalent without N+1 queries.

8.2 Caching

No explicit caching layer (Redis / in-memory) is implemented. Any caching strategy would be additive — the Facade (UserModelService) and BaseService are the natural integration points.

8.3 Load Balancing

The service is stateless with JWT auth stored in HTTP-only cookies — no server-side sessions. This makes it horizontally scalable behind a load balancer, with MongoDB as shared state.

8.4 Fault Tolerance and Failure Modes

- **Startup failure:** DatabaseService.connect() throws; index.ts catches and calls process.exit(1) to trigger process-manager restarts.
- **Error normalisation:** asyncHandler forwards all unhandled promise rejections to errorHandler, which returns structured JSON without leaking stack traces in production.
- **Token expiry:** auth.middleware distinguishes TokenExpiredError (403) from JsonWebTokenError (401) — callers get actionable status codes.
- **KeepAliveService:** catches ping failures gracefully — logs but does not crash the server.

8.5 Concurrency / Async Handling

All database and external-API calls are awaited per request. asyncHandler ensures rejected promises are forwarded to Express error middleware rather than causing unhandled-rejection crashes. Long-running operations (Cloudinary uploads) block the handler but not other concurrent requests (Node event-loop model).

9 Question & Answer Section

9.1 Beginner Level

Q: Is this a monolith or microservices?: It is a layered monolith — a single Express app created by AppFactory and started by index.ts.

Q: Where is authentication handled?: JWT Bearer / cookie auth is in `src/middlewares/auth.middleware.js`. It attaches `req.user` on protected routes. An optional variant (`optionalAuth.middleware.js`) attaches user only if a valid token is present.

Q: Where do errors go when a controller throws?: `asyncHandler` catches the rejected promise and calls `next(error)`. Express routes it to `errors.middleware.ts`, which serialises it as a structured JSON response.

Q: How are files uploaded?: Multer middleware (`multer.middleware.js`) handles disk storage. The file path is then passed to `uploadOnCloud()` in `src/utlis/cloudinary.js`, which uploads to Cloudinary CDN.

9.2 Intermediate Level

Q: What ensures only one MongoDB connection is created?: `DatabaseService` uses the Singleton pattern — a private constructor and a static `getInstance()` that returns the same instance on every call.

Q: How does BaseService help concrete services?: `BaseService` provides generic `create`, `findById`, `find`, `updateById`, `deleteById` using a Mongoose Model injected via the constructor (DIP). Concrete services extend it and add domain-specific methods without duplicating CRUD boilerplate.

Q: How does the aggregation pipeline avoid N+1 queries for channel profiles?: `getUserChannelProfile` uses a single `User.aggregate([...])` call with `$lookup` stages to join subscriptions, `$addFields` to compute counts, and `$project` to shape the output — one round-trip to MongoDB instead of one per subscriber.

Q: What is the benefit of asyncHandler over try/catch in every controller?: `asyncHandler` is an Adapter + Strategy. It defines the `catch+next` forwarding once. Every controller becomes a pure async function; error plumbing is not repeated. This is a DRY + SRP win.

9.3 Advanced Level

Q: What scaling issues exist with the current Cloudinary upload flow?: `uploadOnCloud()` is awaited synchronously inside the HTTP request handler. Large files block the handler for the duration of the upload. The fix: move uploads to a background job queue (`Bull/BullMQ`), return a job ID immediately, and notify the client when the upload completes.

Q: Where is there architectural drift?: Controllers (user.controller.js etc.) are still in JavaScript while base classes and models are TypeScript. This creates a mixed-type boundary where TypeScript compiler cannot enforce contracts on the JS files. Migration to .ts is in progress.

Q: How would you add Redis caching without touching controllers?: Wrap UserModelService (Facade) with a caching decorator or create a CachedUserModelService that extends or composes UserModelService. Because controllers depend on the service abstraction (DIP), swapping in the cached version only requires changing the instantiation site.

10 Strengths & Weaknesses

10.1 Strengths

- **Explicit OOP base classes:** BaseController, BaseRoute, BaseService, BaseError enforce a consistent contract for all contributors.
- **True Singleton enforcement:** DatabaseService and KeepAliveService cannot be double-instantiated — protected by private constructors.
- **asyncHandler eliminates boilerplate:** zero try/catch blocks needed in controllers; all error routing is centralised.
- **LSP-correct error hierarchy:** Object.setPrototypeOf() ensures instanceof checks work after TypeScript compilation.
- **MongoDB aggregation pipelines:** channel-profile and subscription queries are efficient single-trip aggregations.
- **Token rotation strategy:** separate access/refresh token lifetimes with cookie-based secure storage.

10.2 Weaknesses / Improvement Opportunities

- **JS/TS mixed controllers:** controllers are .js while base classes are .ts, losing TypeScript's type-checking at the most important layer. Completing the migration would catch type errors at compile time.
- **No service layer for most controllers:** controllers call Mongoose models directly rather than through a service abstraction. This makes unit testing harder (no seam to inject a mock). Following UserModelService's Facade pattern for VideoService, SubscriptionService would fix this.
- **Synchronous file uploads block the event loop:** Cloudinary uploads are awaited inside HTTP handlers. Background job queues would improve tail latency.
- **No input validation layer:** validation is done with ad-hoc if-checks in controllers. Adding Zod or express-validator at the route boundary would centralise validation and produce uniform error messages.
- **No refresh-token rotation on use:** the current flow issues a new refresh token on login but not on every refresh call. Implementing rotation-on-use would prevent refresh-token replay attacks.

10.3 Refactoring Suggestions (Low Risk)

1. Convert controller .js files to .ts to get full type-safety on req/res.
2. Create VideoService and SubscriptionService following UserModelService's Facade pattern.
3. Add Zod schemas at route boundaries for unified input validation.
4. Move Cloudinary uploads to a BullMQ job queue with a webhook callback.

5. Add refresh-token rotation on every /refresh-token call to mitigate replay attacks.

11 Conclusion

JS_Backend (Reelify) demonstrates a well-structured, OOP-driven layered architecture with deliberate application of SOLID principles and multiple classical design patterns. The Singleton, Factory Method, Template Method, Strategy, Adapter, Facade, and Observer patterns are all present and mapped to concrete files. The largest quality gains would come from completing the TypeScript migration of controllers, adding a service abstraction layer for Video and Subscription, and introducing a background job queue for file uploads to improve scalability and testability.